

📄 Análise Detalhada: Tecnologias e Técnicas do Bulletin Boards UFC

🏠 ARQUITETURA GERAL

- **Aplicação:** Single Page Application (SPA) estática
- **Paradigma:** Frontend-first, orientado a componentes
- **Armazenamento:** Em memória (localStorage não implementado)
- **Tipo:** Progressive Enhancement

🛠️ TECNOLOGIAS FRONTEND

1. HTML5

- **Versão:** HTML5 semântico
- **Charset:** UTF-8
- **Viewport:** Responsivo (meta viewport)
- **Estrutura:**
 - `<header>` com navegação e logo
 - `<main>` com grid layout (conteúdo + sidebar)
 - `<aside>` para sidebar de publicação
 - Uso de `<article>` para cards

2. CSS3

- **Organização:** Modular com SMACSS (Scalable and Modular Architecture for CSS)
- **Arquivos:** 10 arquivos CSS especializados
- **Sistema de variáveis:**
 - Cores (Yellow, Black, White, Blue)
 - Espaçamentos (4px até 48px)
 - Tipografia, borders, transições

Componentes CSS:

├─ base/	
├─ variables.css	(Design tokens: cores, espaçamentos)
├─ reset.css	(Normalização)
└─ typography.css	(Fontes, tamanhos)
├─ components/	
├─ header.css	(Navegação, logo)
├─ cards.css	(Grid 3 cols, layout responsivo)
├─ buttons.css	(Estilos de botões)
├─ modal.css	(Overlay, position: fixed)
└─ forms.css	(Formulários)
└─ responsive.css	(Media queries: 1024px, 720px, 480px)

Recursos CSS Advanced:

- Grid CSS (`grid-template-columns: 1fr 280px`)
- Flexbox
- CSS Variables (`:root`)
- Transições (`--transition: 0.15s ease`)
- Border-radius dinâmico
- Z-index layering (1000 para modal)

3. JavaScript ES6+ Modular

- **Tipo de módulo:** ES6 modules (`type="module"`)
- **Estrutura:** 7 arquivos JS
- **Padrão:** Module Pattern com exports/imports

Arquivos JavaScript:

```
├─ main.js           (Orquestrador principal)
├─ data/
├─   └─ avisos.js    (Array de dados, export)
└─ modules/
    ├── cards.js      (Renderização, ícones SVG)
    ├── filters.js    (Filtros "Últimos" e "Top")
    ├── modal.js      (Modal expansível)
    ├── navbar.js     (Navegação SPA)
    └─ publish.js     (Publicação de avisos)
```

📁 FUNCIONALIDADES IMPLEMENTADAS

1. Sistema de Renderização

- Template literals para HTML dinâmico
- `innerHTML` para injeção de conteúdo
- Renderização reativa em resposta a eventos

2. Filtros e Busca

```
- Filtro "Últimos": Ordena por data (DESC)
- Filtro "Top": Ordena por curtidas (DESC)
- Busca em tempo real (input event listener)
```

3. Modal Interativo

- Criação dinâmica de `<div>` modal
- `position: fixed` com overlay
- Fechamento por: botão "X", click no overlay, Escape
- `document.body.style.overflow = 'hidden'` para bloquear scroll

4. Navegação SPA (Single Page Application)

- Links de navegação sem reload
- `data-page` attributes
- Event listener em `click` dos links
- Eventos customizados (`window.dispatchEvent('page-changed')`)

5. Sistema de Likes e Views

- Mutação de estado no array `avisos`
- Toggle de classe `.liked` para ícones preenchidos
- SVG dinâmicos para ícones (eye, thumbs up)

6. Data e Formatação

```
- Parsing: new Date(dateString)
- Formatação: toLocaleDateString('pt-BR', {day, month, year})
```

🎨 DESIGN E ESTILOS

Paleta de Cores

```
--yellow: #FFE600      (Destaque principal)
--yellow-soft: #FFEC33  (Backgrounds suaves)
--black: #1a1a1a        (Texto principal)
--white: #ffffff         (Fundo)
--blue: #2563eb          (CTA, interações)
--gray: #555, #333, #999 (Variações)
```

Tipografia

- Responsive font sizing
- Cores de texto mutáveis
- Sistema de pesos implícito

Layout Responsivo

```
Desktop (>1024px):  3 colunas de cards
Tablet (≤1024px):  2 colunas + sidebar abaixo
Mobile (≤720px):   1 coluna + toolbar compacta
Phones (≤480px):   Otimizado extremo
```

📌 PADRÕES E TÉCNICAS

1. Event Delegation

```
attachCardClickListeners() // Click em cards
data-action="like|view"    // Botões específicos
```

2. Closures e Contexto

```
handleFilterChange(filter) // Função que captura currentFilter
```

3. Spread Operator e Destructuring

```
const { filtered, sorted } = getFilteredAndSorted()
[...filtered].sort()         // Cópia para não mutar original
```

4. Array Methods

- `.filter()` - Busca
- `.sort()` - Ordenação
- `.map()` - Transformação HTML
- `.find()` - Lookup por ID

5. DOM Manipulation

- `document.createElement()` - Modal
- `innerHTML` com `template literals`
- `querySelector()` e `querySelectorAll()`
- Event listeners com arrow functions

📌 RESPONSIVIDADE

Media Queries

```
/* Tablet: 1024px */
.main {
  grid-template-columns: 1fr; /* Sidebar embaixo */
}

/* Mobile: 720px */
.toolbar { flex-direction: column; }
.cards-grid {
  grid-template-columns: 1fr; /* 1 coluna */
}

/* Pequenos: 480px */
.header { flex-wrap: wrap; }
```

🔗 PERFORMANCE

Otimizações

- CSS modular (carregamento seletivo)
- Template literals pré-compilados
- Eventos delegados (evita múltiplos listeners)
- Transições CSS ao invés de JS

Potencial Melhoria

- LocalStorage para persistência
- Virtual scrolling para muitos cards
- Code splitting

🔗 ESTRUTURA DE DADOS

```
{
  id: 1,
  title: string,
  author: string,
  description: string,
  fullDescription: string,
  views: number,
  likes: number,
  date: "YYYY-MM-DD",
  liked: boolean,
  tags: string[]
}
```

🔗 FUNCIONALIDADES PRINCIPAIS

Funcionalidade	Tecnologia	Localização
Renderização de cards	Template literals + innerHTML	cards.js
Modal expansível	DOM manipulation + CSS fixed	modal.js
Navegação SPA	Event listeners + DOM swap	navbar.js
Filtros	Array.sort() + re-render	filters.js
Busca	String.includes() em tempo real	main.js
Like/View	Mutação de estado + toggle CSS	cards.js
Publicação	Modal form + event dispatch	publish.js

🔗 SEGURANÇA (Notas)

- Sem sanitização de HTML (XSS potencial)
- Sem CORS (SPA local)
- Sem autenticação (frontend only)
- Dados públicos em memória